Trabajo académico RabbitMQ

Resultado y alcance

Diez casos RabbitMQ documentados como Cumple, conclusión propia y no evaluación del profesor. Este PDF consolida la parte RabbitMQ requerida por la guía, no la entrega global de veinte casos ni el experimento comparativo Kafka. Objetivo: verificar integración HTTP, mensajería y consumo mediante evidencia histórica genuina.
Generación documental UTC: 2026-11-01T13:17:10.352569+00:00. Ensayos: 2026-10-27 UTC; no se repitieron.
No se inventan integrantes, grupo, fecha de entrega, licencia ni evaluación docente.

Código de destino y navegación: repositorio GitHub.
La URL expresa el destino previsto, no acredita push ni acceso remoto verificado.
Rutas conservadas desde la raíz: rabbitmq/ y evidencias/rabbitmq/.
El PDF convierte enlaces a GitHub blob/main; el Markdown mantiene enlaces relativos.
No se publica el PDF original del profesor.

Arquitectura e infraestructura

Diagrama propio funcional - no GUI observada

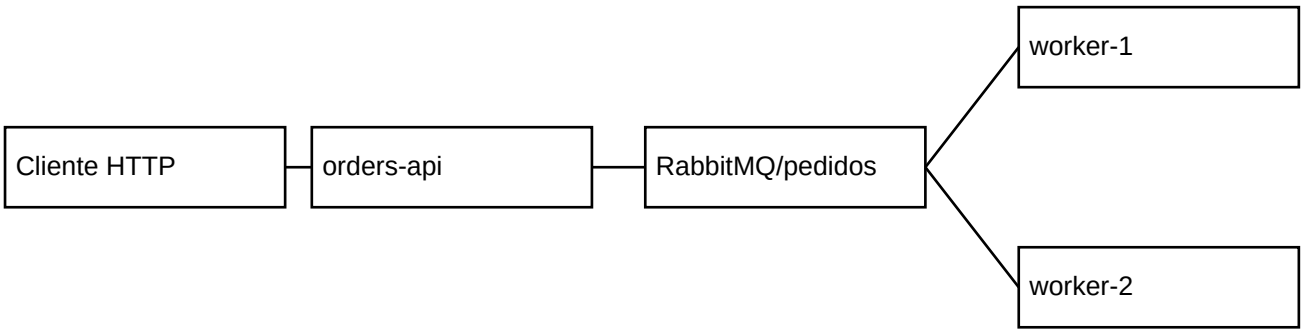


Diagrama propio funcional; no GUI observada.

Diagrama propio funcional, no captura de GUI observada. Cliente HTTP -> API -> broker RabbitMQ / cola pedidos -> worker-1 o worker-2. Los workers compiten por mensaje: no broadcast. Cuatro contenedores independientes: una API, un broker, dos workers.
HTTP POST /orders en 127.0.0.1:3000; AMQP TCP en 127.0.0.1:5672; Management HTTP en 127.0.0.1:15672. Dos réplicas comparten pedidos. Fuente de estado histórico: Compose ps.
Diagrama vectorial propio: SVG.

Node.js/Express/amqplib; canal de confirmación del productor, cola durable=true, publicación persistent=true, messageId=correlationId=orderId. El worker usa ack manual (noAck=false), prefetch(1) y demora simulada PROCESSING_DELAY_MS default 3000 ms antes

del ack. Configuración y fragmentos literales se incluyen en CP-08; los intervalos observados no prueban un SLA ni configuración efectiva exacta de 3 s.
El ack confirma consumo, no una notificación externa real. Durabilidad/persistencia no equivalen a HA ni exactly-once. RabbitMQ 4.4.0 consta en CP-06; etiqueta histórica rabbitmq:4-management en Compose no fija por sí sola la versión ejecutada.

Entorno, metodología y trazabilidad

Los estados de Docker/Management, versiones y horas citados son históricos; no son verificación de la máquina Windows destino ni de Hyper-V. La reconstrucción PDF es Python opcional; Node no depende de Python. Ruta Windows opcional, sin WSL:

```
py -m venv .venv
.venv\Scripts\python -m pip install -r tools/requirements-informe.txt
.venv\Scripts\python tools/generar_informe.py
```

Este generador no ejecutó Docker, Postman, curl ni solicitudes. La verificación Start/Test/Stop registró PASS en 77 s en proyecto aislado, 2/2 tests y npm con cero vulnerabilidades; conservó los cuatro IDs originales y el volumen aislado al detener.
Ese diagnóstico utilizó PowerShell 7.6.9 en Linux, Node 26.8.1 en el host, Docker Engine 29.6.2 y Node 24 dentro de los contenedores:
valida allí los scripts PowerShell, no una máquina Windows/Hyper-V nativa. La validación nativa Windows/Hyper-V sigue sin realizarse. Este resultado está documentado en docs/entrega/VERIFICACION_ENTREGA.md y es separado de las pruebas offline del generador. No se atribuyen solicitudes nuevas a los casos históricos.

Se incorporan íntegros los once campos de cada reporte. Logs raw y comandos enlazados permanecen en evidencias/rabbitmq; no se copian cientos de líneas sin contexto al PDF.
Las siete capturas se incluyen al final, una por página con orientación adecuada.
Seis se muestran completas; CP-12 muestra un detalle identificado con enlace al original íntegro. Capturas CLI, GUI y vistas derivadas se identifican por separado.

Familias de IDs: b7a74952 (CP-01/02/03/07/09/11, una publicación compartida); 8d488fb8 (CP-06/08, traza nativa); seis IDs exclusivos de CP-10 (tabla enlazada); ea298437 (CP-12). UUID completos figuran en cada caso. No mezclar los experimentos.
En CP-06 published/received nativos del broker preceden al log API order_published: el log API se escribe tras confirmación. received nativo significa entrega, no ack.
El broker registra IDs mediante rabbitmq_tracing, no mediante su stdout operativo habitual; se conservaron JSONL/payload y comprobación de restauración del diagnóstico.

Matriz de cobertura RabbitMQ

| Caso | Estado documental propio |

| --- | --- |

| CP-01 | Cumple |

| CP-02 | Cumple |

| CP-03 | Cumple |

| CP-06 | Cumple |

| CP-07 | Cumple |

| CP-08 | Cumple |

| CP-09 | Cumple |

| CP-10 | Cumple |

| CP-11 | Cumple |

| CP-12 | Cumple |

Reportes completos: once campos por caso

CP-01 — Creación de pedido vía API — RabbitMQ

- ID del caso: CP-01; revisión documental de corrida 03.

- Arquitectura: RabbitMQ; orders-api -> pedidos -> notification-worker.

- Objetivo: Validar que el endpoint recibe y responde correctamente.

- Precondiciones: API y broker disponibles; workers apagados. baseline.

- Datos de entrada: Corrida 03 del 2026-10-27; un POST /orders con JSON {"customerName":"Pedro","product":"Libro de arquitectura","quantity":1}. Pedido ORD-b7a74952-2957-49cb-a5db-f88d7b2ce443; correlationId igual al orderId.

- Pasos ejecutados:

1. Se envió una sola solicitud válida mediante Postman CLI (request y respuesta CLI).

2. Se conservaron respuesta, ID y aserciones; revisión actual exclusivamente offline.

- Resultado esperado: Endpoint recibe JSON y responde aceptación con orderId.

- Resultado obtenido: HTTP 202 Accepted, 27 ms, status accepted, mensaje «Pedido recibido y enviado a RabbitMQ», ID indicado; 4/4 aserciones y una request. request y respuesta CLI.

- Evidencia (capturas): Captura real de Postman CLI, aportada por el usuario: muestra POST localhost:3000/orders, datos de Pedro, HTTP 202 Accepted, 27 ms, status accepted, mensaje esperado y el orderId completo indicado, partido visualmente en líneas. Es una captura de la salida CLI original guardada de corrida 03, no de la GUI de Postman ni de una corrida nueva; se contrasta con el stdout original. Management pendiente acredita almacenamiento posterior, no request/respuesta HTTP.

- Estado: Cumple.

- Análisis técnico: Solicitud, respuesta y captura real de la salida CLI guardada respaldan la aceptación HTTP del mismo pedido de corrida 03. No se afirma una captura GUI de Postman ni una ejecución nueva. No se ha observado un defecto de API. Conclusión documental propia, no calificación del profesor. Fuente: guía pp. 4 y 7–13 y reporte original CP-11. Sin ejecución live ni modificación de evidencias originales.

CP-02 — Envío de mensaje a RabbitMQ — RabbitMQ

- ID del caso: CP-02; revisión documental de corrida 03.

- Arquitectura: RabbitMQ; orders-api -> pedidos -> notification-worker.

- Objetivo: Verificar que el pedido genera un mensaje en la cola.

- Precondiciones: API/broker disponibles; ambos workers apagados, Consumers=0 y cola vacía: baseline.

- Datos de entrada: Corrida 03 del 2026-10-27; un POST /orders con JSON {"customerName":"Pedro","product":"Libro de arquitectura","quantity":1}. Pedido ORD-b7a74952-2957-49cb-a5db-f88d7b2ce443; correlationId igual al orderId.

- Pasos ejecutados:

1. Se verificó baseline vacío.

2. Se publicó un único pedido (request y respuesta CLI).

3. Se conservaron log API, cola pendiente y captura con el pendiente.

- Resultado esperado: El pedido genera un mensaje en pedidos.

- Resultado obtenido: order_published a 2026-10-27T01:39:39.627Z, queue pedidos, mismo ID. Ready 0 -> 1, Unacked=0, Total=1, Consumers=0; durable=true. log API; cola pendiente.

- Evidencia (capturas): Management pendiente: Ready=1, Unacked=0, Total=1, Consumers=0, Persistent=1, durable=true. No muestra ID ni payload.

- Estado: Cumple.

- Análisis técnico: Baseline vacío, publicación única y aumento de un mensaje con cero consumidores respaldan el envío a cola exigido por CP-02. No se inspeccionó payload ni se usó Get messages. Esta correlación experimental no sustituye el log del broker por ID exigido en CP-06. Conclusión documental propia, no calificación del profesor. Fuente: guía pp. 4 y 7–13 y reporte original CP-11. Sin ejecución live ni modificación de evidencias originales.

CP-03 — Consumo de mensaje RabbitMQ — RabbitMQ

- ID del caso: CP-03; revisión documental de corrida 03.

- Arquitectura: RabbitMQ; orders-api -> pedidos -> notification-worker.

- Objetivo: Validar que el consumidor procesa el mensaje.

- Precondiciones: Pedido de corrida 03 almacenado con workers apagados (cola pendiente); después se iniciaron ambos sin nueva publicación.

- Datos de entrada: Corrida 03 del 2026-10-27; un POST /orders con JSON {"customerName":"Pedro","product":"Libro de arquitectura","quantity":1}. Pedido ORD-b7a74952-2957-49cb-a5db-f88d7b2ce443; correlationId igual al orderId.

- Pasos ejecutados:

1. Se conservó el pendiente.

2. Se iniciaron los workers una sola vez (registro histórico en el reporte CP-11).

3. Se guardaron logs worker-2 y resultado final.

- Resultado esperado: El consumidor recibe y procesa el mensaje pendiente.

- Resultado obtenido: worker-2 / aea9d0d7aa00 registra order_received 2026-10-27T01:46:31.200Z y order_processed 2026-10-27T01:46:34.229Z del mismo ID; duración 3.029 s. Final Ready=0, Unacked=0, Total=0, Consumers=2. logs worker-2; resultado final.

- Evidencia (capturas): Docker Desktop Logs worker-2 contiene el ID de esta corrida recibido/procesado a 01:46:31/34, antes del lote CP-10; Management final muestra cola 0/0/0 y Consumers=2.

- Estado: Cumple.

- Análisis técnico: Los logs con ID y captura real del worker demuestran procesamiento; la cola final corrobora ausencia de pendientes en este escenario. No se promete exactly-once ni ausencia general de pérdida. No se confunden las tres líneas posteriores por instancia del lote CP-10 con este pedido. Conclusión documental propia, no calificación del profesor. Fuente: guía pp. 4 y 7–13 y reporte original CP-11. Sin ejecución live ni modificación de evidencias originales.

CP-06 — Trazabilidad del flujo — RabbitMQ

- ID del caso: CP-06; corrida de trazabilidad 01, 2026-10-27 (UTC).

- Arquitectura: RabbitMQ; orders-api -> broker RabbitMQ / cola pedidos -> notification-worker, con dos instancias activas.

- Objetivo: Seguir el mismo orderId / correlationId en los registros de API, broker y consumidor, como exige la guía p. 4.

- Precondiciones: API y broker healthy, dos workers running, Consumers=2, cola vacía y durable=true. RabbitMQ 4.4.0; plugin de diagnóstico inicialmente deshabilitado y Firehose apagado. Estado inicial. Activación temporal autorizada para esta prueba local; sin cambios de aplicación o Compose.

- Datos de entrada: Un POST /orders con JSON {"customerName":"Pedro","product":"Libro de arquitectura","quantity":1}. Pedido ORD-8d488fb8-62c9-4e37-935d-31aed2a8aa9e; correlationId igual al orderId. Es una corrida distinta de la usada para CP-01/02/03/11.

- Pasos ejecutados:

1. Se conservaron versiones, salud, plugins, vhost, estado de cola y consumidores.

2. Se habilitó temporalmente el plugin nativo rabbitmq_tracing online y se creó una traza propia con formato JSON, patrón # y vhost /; se comprobó que su consumidor estaba listo antes de publicar.

3. Se ejecutó una sola solicitud Postman CLI: HTTP 202, 1655 ms, 4/4 aserciones y exitcode 0. Comando y metadatos; stdout original.

4. Se exportó el archivo nativo del broker y se conservaron logs API/worker del mismo ID; se verificaron message_id, correlation_id y orderId del payload decodificado.

5. Se eliminó solo la traza propia y se apagó Firehose. Tras interrumpirse la ejecución, una inspección independiente verificó que el plugin ya estaba deshabilitado y que el estado inicial estaba restaurado, sin volver a publicar ni reiniciar servicios.

- Resultado esperado: El mismo pedido identificable en registros reales de API, broker y consumidor, con evidencias legibles y restauración del diagnóstico temporal.

- Resultado obtenido: Correlación técnica verificada para el ID indicado. Broker nativo: published a 02:42:35.087+00:00, con routing_keys/routed_queues pedidos; received a 02:42:35.103+00:00, queue pedidos. Ambos registros incluyen properties.correlation_id y properties.message_id iguales al ID; el payload base64 contiene el mismo orderId. API order_published 02:42:35.123Z. Worker-1 / 9daaf02d21e3: order_received 02:42:35.158Z, order_processed 02:42:38.411Z, 3.253 s de procesamiento observado. Todo el 2026-10-27, UTC. JSONL nativo, log API, log worker y correlación derivada validada. Estado final verificado: dos workers, API/broker healthy, Consumers=2, Ready/Unacked/Total=0, plugins iniciales restaurados y Firehose apagado: verificación.

- Evidencia (capturas): Captura real aportada por el usuario y revisada completa de API, BROKER NATIVO y WORKER-1 del mismo ID. Captura de VISTA_EVIDENCIAS.txt, explícitamente DERIVADA de fuentes originales reales, no GUI ni JSONL nativo modificado. Fuentes: JSONL original, log API y log worker, conservados intactos. No mezclar con IDs de otros casos.

- Estado: Cumple; conclusión propia, no calificación del profesor. Cierre exclusivamente documental, sin nueva ejecución.

- Análisis técnico: El registro del broker procede de su plugin nativo de diagnóstico, no de una línea inventada de la aplicación ni de contadores de Management. Sus logs operativos habituales no registraron este ID: filtro vacío conservado. En el formato nativo received representa entrega desde el broker, no ack ni procesamiento completo; esto último se acredita por el evento del worker. El log API puede aparecer después del evento del broker porque se escribe tras confirmar la publicación. La traza tiene coste de recursos y registra payload; no sirve para extrapolar rendimiento, exactly-once o recuperación ante caídas. La interrupción dejó sin metadatos finales al script de ensayo: no se inventa exitcode para la deshabilitación del plugin; se acredita la restauración mediante inspección independiente. Tres tests offline del formato pasaron; enlaces, IDs, etapas y limpieza se verificaron. Conclusión

propia, no calificación del profesor. Resumen completo, límites, comandos y fuentes oficiales; fuentes normativas: guía p. 4 y Anexo 1 pp. 7–13. La evidencia previa de corrida 03 permanece intacta, pero ya no es la base principal de este reporte.

CP-07 — Responsive — RabbitMQ

- ID del caso: CP-07; revisión documental de corrida 03, 2026-10-27; ORD-b7a74952-2957-49cb-a5db-f88d7b2ce443.

- Arquitectura: RabbitMQ; orders-api -> cola pedidos -> notification-worker.

- Objetivo: demostrar que la API responde sin esperar al consumidor (guía p. 4), sin imponer un SLA ni un umbral de milisegundos.

- Precondiciones: API y broker disponibles; ambos workers detenidos; baseline con Consumers=0 y Ready/Unacked/Total=0. Es un escenario restringido de consumidor ausente, válido para el requisito; no una prueba bajo carga.

- Datos de entrada: un POST http://localhost:3000/orders, JSON {"customerName":"Pedro","product":"Libro de arquitectura","quantity":1}. Un único pedido, correlationId igual al orderId anterior.

- Pasos ejecutados:

1. En la corrida original se detuvieron ambos workers y se verificó el baseline vacío; procedimiento y comandos conservados.

2. Se envió una solicitud por Postman CLI, guardando stdout original y exitcode.

3. Se verificó publicación del mismo ID en API y cola pendiente, antes de reactivar consumidores.

4. Tras la captura pendiente, se iniciaron los workers sin otro pedido; se registraron recepción y procesamiento del mismo ID y estado final.

5. Esta unidad solo revisó archivos existentes; no repitió requests ni intervino servicios.

- Resultado esperado: respuesta HTTP de aceptación antes del procesamiento del consumidor; el trabajo puede permanecer pendiente y completarse después.

- Resultado obtenido: HTTP 202 Accepted, status: accepted, 27 ms, 4/4 aserciones y exitcode 0 con workers apagados. El pedido permaneció Ready=1, Unacked=0, Total=1, Consumers=0. Aplicación UTC del 2026-10-27: publicación 01:39:39.627Z; recepción 01:46:31.200Z; procesamiento 01:46:34.229Z, worker-2 / aea9d0d7aa00. Publicación->recepción: 411.573 s; recepción->procesamiento: 3.029 s. Por tanto la aceptación HTTP no esperó a que el consumidor, iniciado varios minutos después, terminara. Cola final 0/0/0 con Consumers=2.

- Evidencia (capturas):

- HTTP 202 y 27 ms, pedido completo: captura real de salida CLI guardada, no GUI de Postman ni ejecución nueva.

- Management pendiente: Ready=1 y Consumers=0.

- Docker Desktop Logs worker-2: mismo pedido a 01:46:31/34, antes de los seis pedidos de CP-10; no se usan estos últimos.

- Management final: cola vacía y Consumers=2. Los contadores no muestran ID; correlación por baseline vacío, publicación única y logs.

- Estado: Cumple. Conclusión propia sobre el requisito observado, no calificación del profesor.

- Análisis técnico: se eligió corrida 03 por tener HTTP y capturas reales de las fases pendiente/procesada con el mismo ID. No se mezclan datos de trazabilidad corrida 01 ni se comparan sus 1655 ms (diagnóstico activo) como benchmark. Los 27 ms son duración HTTP medida por CLI, no una hora absoluta de fin. El campo HTTP_EVIDENCE contiene un timestamp de registro posterior a respuesta, no se equipara a fin exacto HTTP; Date tiene precisión de segundos y order_published tampoco es fin HTTP. No se calcula HTTP-fin->procesamiento con esos sustitutos. La secuencia workers ausentes->respuesta aceptada/pedido pendiente->reactivación posterior demuestra el orden requerido sin necesitar esa precisión. Las diferencias arriba se calculan solo entre timestamps de aplicación de sus eventos, no prefijos Docker ni hora local de capturas. El worker espera PROCESSING_DELAY_MS (default 3000) mediante temporizador antes del ack; 3.029 s es intervalo observado, no prueba de configuración efectiva exacta ni SLA de 3 s. Esta evidencia demuestra independencia temporal de la

respuesta en este escenario; no latencia bajo carga, exactly-once, ausencia general de pérdida ni CP-08/09/12. Originales intactos, sin capturas fabricadas.

CP-08 — Message-driven — RabbitMQ

- ID del caso: CP-08.

- Arquitectura: RabbitMQ.

- Objetivo: comprobar comunicación API–worker mediada por el broker, sin llamadas directas entre esos servicios en la implementación inspeccionada (guía del taller p. 4 y Anexo 1 pp. 7–13).

- Precondiciones: implementación existente Node.js/Express/amqplib; API, broker y workers independientes. Se reutiliza únicamente trazabilidad/corrida-01 de CP-06, ejecutada el 2026-10-27. Diagnóstico temporal ya desactivado y dos workers restaurados según verificación y estado conservado; no se consulta ni altera el runtime aquí.

- Datos de entrada: pedido original {"customerName":"Pedro","product":"Libro de arquitectura","quantity":1}; mensaje ProcessOrder, createdAt 2026-10-27T02:42:34.616Z. Único orderId/correlationId/messageId: ORD-8d488fb8-62c9-4e37-935d-31aed2a8aa9e.

- Pasos ejecutados:

1. Leer completos api.mjs, broker.mjs, worker.mjs, docker-compose.yml y package.json; examinar publicación, conexión y consumo, no solo nombres.

2. Contrastar configuración y fragmentos exactos de abajo: ambos procesos conectan por AMQP a RabbitMQ; el productor publica en la cola que consume el worker.

3. Revisar el JSONL nativo original, decodificar sus dos payloads base64 y contrastar IDs con los logs originales de API y worker.

4. Revisar la captura real existente de la vista derivada, sin producir otra solicitud, captura sintética ni reactivar tracing.

- Resultado esperado: código/configuración con publicación y suscripción AMQP, más evidencia nativa del broker que relaciona publicación y entrega del mismo pedido al consumidor; sin invocación HTTP API–worker en este flujo.

- Resultado obtenido: broker published a 2026-10-27T02:42:35.087+00:00, exchange "" (predeterminado), routing_keys y routed_queues ["pedidos"]; broker received a 02:42:35.103+00:00, queue pedidos. En ambos registros correlation_id y message_id coinciden con payload.orderId. API registra order_published 02:42:35.123Z; worker-1 / 9daaf02d21e3 registra order_received 02:42:35.158Z y order_processed 02:42:38.411Z. Son marcas UTC de fuentes distintas: el log API ocurre tras la confirmación, no antes de publicar en el broker. La mediación observada se acredita por la traza nativa, no por la mera coexistencia de logs o contenedores.

- Evidencia (capturas): captura real aportada por el usuario, revisada completa: API, BROKER NATIVO y WORKER-1 del mismo ID. Está titulada CP-06 y VISTA DERIVADA DE REGISTROS REALES; se reutiliza por pertinencia al flujo CP-08. No es GUI de RabbitMQ ni JSONL nativo modificado. Vista textual conservada. Fragmentos verificables y fuentes enlazadas abajo complementan la captura.

- Estado: Cumple.

- Análisis técnico: conclusión propia, no calificación del profesor. En el código inspeccionado la API importa broker.mjs y envía el mensaje mediante sendToQueue; el worker importa el mismo módulo de conexión y consume pedidos por AMQP. No existe llamada directa API–worker en ese flujo: el worker no expone servidor HTTP y la API no lo invoca. El fetch del healthcheck Compose consulta la propia API en localhost, no al worker. Docker por sí solo no demuestra esta propiedad. La cola compartida representa consumidores competidores, no broadcast ni tópico Kafka. received nativo significa entrega desde el broker, no ack; el código acredita ack manual tras la demora simulada, sin afirmar una traza de ack por ID. Durabilidad/persistencia y confirmación del productor no prueban exactly-once, recuperación CP-12 ni resiliencia CP-09. La muestra es un pedido; HTTP 202/1655 ms y 4/4 aserciones de la corrida original son contexto complementario, no benchmark CP-07. No se mezclan corrida b7a ni lote CP-10.

Código/configuración inspeccionados — fragmentos literales

api.mjs, líneas 5–9: canal de confirmación y cola durable.

```
const connection = await connect();
const channel = await connection.createConfirmChannel();
channel.on('error', error => log('channel_error', { error: error.message }));
channel.on('close', () => process.exit(1));
await channel.assertQueue(queue, { durable: true });
```

api.mjs, líneas 26–30: publicación persistente correlacionada; callback confirma el broker, no el procesamiento del worker.

```
    await new Promise((resolve, reject) => channel.sendToQueue(
      queue, Buffer.from(JSON.stringify(message)),
      { persistent: true, contentType: 'application/json', messageId: orderId,
correlationId: orderId },
      error => error ? reject(error) : resolve(),
    ));
```

broker.mjs, líneas 1–3 y 14 respectivamente: cliente y nombre de cola constantes; variable real AMQP_URL, no RABBITMQ_URL ni RABBITMQ_QUEUE.

```
import amqp from 'amqplib';

export const queue = 'pedidos';
      const connection = await amqp.connect(process.env.AMQP_URL);
```

worker.mjs, líneas 9–11, 17–21 y 27 respectivamente: consumo de la misma cola, prefetch 1 y ack manual después de la simulación.

```
await channel.assertQueue(queue, { durable: true });
await channel.prefetch(1);
await channel.consume(queue, async message => {
    log('order_received', trace);
    await new Promise(resolve => setTimeout(resolve, delay));
    // Notificación simulada. El ack se envía solo al terminar, no al recibir.
    channel.ack(message);
    log('order_processed', trace);
}, { noAck: false });
```

docker-compose.yml, líneas 20–25 y 43–51: procesos separados. Ambos AMQP_URL señalan al servicio rabbitmq:5672; se omiten aquí valores de conexión locales, consultables en la fuente original. Dos réplicas de worker configuradas. package.json declara Express y amqplib; no constituye por sí mismo prueba runtime. La revisión de estos archivos no los modifica.

Validación y límites de esta unidad

Revisión documental pasiva, sin TDD de aplicación ni nuevos tests/scripts de producción. Resumen y comprobación offline. Los originales y CP-07 se preservan; no se ejecutan Docker, Postman, curl, reinicios, instalaciones ni peticiones nuevas. El estado de servicios citado es histórico, no una verificación live actual.

CP-09 — Resilient — RabbitMQ

- ID del caso: CP-09; corrida 03, 2026-10-27; ORD-b7a74952-2957-49cb-a5db-f88d7b2ce443.

- Arquitectura: RabbitMQ; orders-api -> cola pedidos -> notification-worker (dos instancias locales).

- Objetivo: comprobar que la indisponibilidad simulada del consumidor no impide aceptar/publicar pedidos en la API y que el pedido pendiente se procesa cuando regresa el consumidor. Guía p. 4: «El sistema soporta caída del consumidor»; no exige kill ni interrupción durante procesamiento para CP-09.

- Precondiciones: API/broker running y healthy, workers inicialmente running (Compose inicial). Tras detenerlos, baseline durable=true, Consumers=0, Ready/Unacked/Total=0/0/0. Dos workers detenidos y un pedido son parámetros seleccionados de este ensayo, no mínimos impuestos por el profesor.

- Datos de entrada: un POST /orders, application/json, {"customerName":"Pedro","product":"Libro de arquitectura","quantity":1}; request/respuesta originales.

- Pasos ejecutados:

1. Se comprobó el estado inicial y se simuló caída mediante stop controlado: docker compose -p qa-taller-integracion-rabbitmq stop notification-worker, de 01:39:19.324175 a 01:39:30.165185 UTC, exitcode del comando 0. Compose tras stop: ambos exited, ExitCode de contenedor 137; API/broker healthy. No se confunde ese 137 con el éxito del comando ni se afirma parada graciosa.

2. Tras espera acotada de Consumers=0 y baseline vacío, se publicó una única solicitud con Postman CLI, exitcode 0. No hubo otra publicación para documentar este caso.

3. Se verificó order_published del mismo ID, cola pendiente y Compose durante la demostración. El snapshot histórico, a 01:39:45.394411+00:00, conserva ambos workers exited y Consumers=0. No describe el estado actual.

4. Después de la captura pendiente, se verificaron precondiciones y se ejecutó una sola vez start notification-worker, exitcode 0, sin nueva publicación. Se esperó como máximo 45 s y se registraron logs del mismo pedido y resultado final.

- Resultado esperado: ante consumidor no disponible, la API sigue aceptando/publicando; el broker retiene el pedido pendiente, que se procesa al regresar el consumidor. No se requiere procesamiento inmediato mientras está detenido.

- Resultado obtenido: HTTP 202 Accepted en 27 ms, una request, 4/4 aserciones, exitcode 0. API/broker healthy antes y durante la demostración; publicación 2026-10-27T01:39:39.627Z con ambos workers detenidos. Cola durable: Ready=1, Unacked=0, Total=1, Consumers=0; captura Persistent=1. El mismo ID fue recibido 01:46:31.200Z y procesado 01:46:34.229Z, worker-2 / aea9d0d7aa00, redelivered=false. Resultado final a 2026-10-27T01:46:40.985636+00:00: ambos workers running, API/broker healthy, Consumers=2, Ready/Unacked/Total=0/0/0. Horas UTC de aplicación, no prefijos Docker ni hora local de UI; 27 ms es observación, no SLA.

- Evidencia (capturas):

- HTTP 202/27 ms y mismo ID: captura real de salida CLI previamente guardada, no GUI de Postman ni nueva ejecución.

- Management pendiente: Ready=1, Total=1, Unacked=0, Consumers=0, Persistent=1, durable=true.

- Management tras regreso: 0/0/0, Consumers=2.

- Docker Desktop worker-2: exclusivamente líneas b7a de 01:46:31/34, anteriores a los seis pedidos CP-10; contrastadas con logs originales.

- Estado: Cumple. Conclusión propia sobre el ensayo documentado, no calificación del profesor.

- Análisis técnico: la caída simulada por stop elimina los consumidores disponibles, pero no bloquea la aceptación/publicación observada. La cola amortigua la indisponibilidad hasta el regreso del worker. Management no muestra ID ni payload: la correlación experimental procede del baseline vacío, publicación única y logs API/worker coincidentes, no de Get messages ni de inspección de payload. CP-11 usa este experimento para almacenamiento y

separación temporal; CP-09 enfatiza continuidad de API ante indisponibilidad y regreso del consumidor. CP-07 analiza respuesta sin esperar consumo. Este experimento no cubre CP-12: aquí no se interrumpió b7a en Unacked antes del ack ni se observó reentrega (redelivered=false); no demuestra recuperación de trabajo en vuelo. La recuperación se verificó después, en una corrida independiente, y quedó cerrada en el reporte CP-12. Se conservó y procesó este ID, sin generalizar «no pérdida», exactly-once, alta disponibilidad, downtime cero, caída del broker, fallo AMQP o capacidad bajo carga. Los intentos 01/02 y sus fallos de observación/instrumentación permanecen en el historial, no se mezclan con esta corrida ni con la traza 8d de CP-06. Cierre exclusivamente documental, sin operaciones runtime ni cambios de aplicación.

CP-10 — Elastic — RabbitMQ

- ID del caso: CP-10, corrida reparto-carga, 2026-10-27 UTC.

- Arquitectura: RabbitMQ; API -> cola pedidos -> dos instancias notification-worker.

- Objetivo: demostrar reparto real del lote entre ambas instancias ya activas.

- Precondiciones: dos workers running; API y broker healthy; Consumers=2, Ready=0, Unacked=0, Total=0. Estado inicial, Compose.

- Datos de entrada: seis POST válidos con Pedro, Libro de arquitectura, quantity=1. Seis es parámetro del laboratorio, no exigencia del profesor.

- Pasos ejecutados:

1. Guardar Compose y Management; validar precondición sin modificar servicios.

2. Ejecutar una sola corrida Postman de seis iteraciones del request válido, sin --output ni login.

3. Esperar hasta 60 s los seis procesamientos y cola vacía con dos consumidores; preservar logs originales antes de derivar resultados.

4. Relacionar IDs y guardar estado final. Comandos y tabla.

- Resultado esperado: seis respuestas accepted con IDs únicos; 24 aserciones correctas; cada pedido publicado, recibido y procesado, con al menos uno procesado por cada worker.

- Resultado obtenido: 6 IDs únicos, 4 x 6 = 24/24 aserciones, todos correlacionados; worker-1 procesó 3 y worker-2 procesó 3. Ready=0, Unacked=0, Total=0, Consumers=2 al cierre. Resultado, tabla.

- Evidencia (capturas): capturas reales de Docker Desktop Logs: worker-1, instancia 9daaf02d21e3, muestra tres order_processed para IDs 962eb263, fbd6453d, e3d243ec; worker-2, instancia aea9d0d7aa00, muestra tres para d047fa00, f83a6ad1, 462f5dc3 (fragmentos de los UUID completos de la tabla). Corresponden al lote del 2026-10-27, 01:58:33–43 UTC, y demuestran directamente reparto entre ambas instancias. Complementan HTTP original, logs nativos, Management final JSON y Compose final; estos últimos son evidencia CLI, no capturas GUI. No se requiere una captura GUI adicional de cola para esta conclusión.

- Estado: Cumple

- Análisis técnico: «Cumple» es la conclusión de nuestro reporte, no una calificación del profesor. El reparto se observa en dos instancias distintas, con los seis IDs aceptados presentes en procesamiento. No se observaron IDs perdidos en este lote; esto no prueba ausencia general de pérdida ni exactly-once. El 3/3 no es requisito; no se midió escalabilidad lineal ni se comparó una contra dos instancias. No se ejecutaron caídas, reinicios, recuperación ni Kafka; aplicación y Compose permanecen sin modificaciones.

CP-11 — Desacoplamiento temporal — RabbitMQ

- ID del caso: CP-11; corrida 03, 2026-10-27. Pedido ORD-b7a74952-2957-49cb-a5db-f88d7b2ce443.

- Arquitectura: RabbitMQ; orders-api -> cola pedidos -> notification-worker, con dos instancias detenidas durante la publicación y reactivadas después.

- Objetivo: comprobar almacenamiento con consumidores ausentes y procesamiento posterior del mismo pedido al iniciarlos, sin nueva publicación.

- Precondiciones: API y broker disponibles; ambos workers detenidos antes del request. Baseline JSON: durable=true, Consumers=0, Ready=0, Unacked=0, Total=0. Las comprobaciones y esperas originales fueron acotadas. Los intentos anteriores y sus fallos se conservan en el registro principal; no forman parte de esta secuencia exitosa.

- Datos de entrada: un único POST http://localhost:3000/orders, Content-Type application/json, body {"customerName":"Pedro","product":"Libro de arquitectura","quantity":1}. Request y respuesta originales.

- Pasos ejecutados:

1. Desde rabbitmq/ del taller se comprobó salud y se detuvieron ambos workers mediante docker compose -p qa-taller-integracion-rabbitmq stop notification-worker (comando literal). Se verificaron workers exited y se esperó Consumers=0, sin mensajes iniciales.

2. Se ejecutó una sola solicitud válida: postman collection run pruebas/RabbitMQ.postman_collection.json -i 'Crear pedido — esperado HTTP 202' --no-report-events --verbose --timeout 30000 --timeout-request 10000 --timeout-script 5000 (comando literal, stdout HTTP, exitcode 0). Se extrajo el ID del resultado guardado, sin otro request.

3. Se verificó Ready=1, Unacked=0, Total=1, Consumers=0, durable=true en cola posterior y resultado previo al consumo; API registró order_published del mismo ID. El usuario aportó la captura del pendiente antes de reactivar.

4. Tras confirmar aún ambos workers exited y el pendiente, se ejecutó una sola vez docker compose -p qa-taller-integracion-rabbitmq start notification-worker (comando literal, exitcode 0). No se publicó otro pedido. Una única espera acotada a 45 s verificó recepción, procesamiento y estado final.

5. Se conservaron logs del mismo ID, Management final y resultado final. El usuario aportó la captura final de cola vacía con dos consumidores.

- Resultado esperado: la API acepta y publica con consumidores apagados; el pedido permanece pendiente en la cola y es procesado después de iniciarlos, correlacionando el mismo ID y los contadores antes/después.

- Resultado obtenido: HTTP 202 Accepted en 27 ms, una request, 4/4 aserciones y exitcode 0. Ambos workers estaban apagados al publicar. Baseline 0/0/0 -> pendiente Ready=1, Unacked=0, Total=1, Consumers=0; captura con Persistent=1. Aplicación en UTC del 2026-10-27: order_published 01:39:39.627Z, order_received 01:46:31.200Z, order_processed 01:46:34.229Z. Recepción y procesamiento por notification-worker-2, instancia aea9d0d7aa00. Transcurrieron 411.573 s entre publicación y recepción, y 3.029 s de procesamiento. A 2026-10-27T01:46:40.985636+00:00, Ready=0, Unacked=0, Total=0, Consumers=2; ambos workers running y API/broker healthy.

- Evidencia (capturas):

- Management: pedido pendiente: Ready=1, Unacked=0, Total=1, Consumers=0, Persistent=1, durable=true.

- Management: pedido consumido: Ready=0, Unacked=0, Total=0, Consumers=2.

- Docker Desktop Logs: worker-2: también contiene las líneas del ID ORD-b7a74952-2957-49cb-a5db-f88d7b2ce443 a 01:46:31/34, anteriores al lote CP-10. Reutilización de evidencia real del mismo pedido; no confundirlo con los seis IDs posteriores de CP-10. Contrastar con los logs filtrados enlazados arriba.

- Estado: Cumple. Conclusión de este reporte respecto del caso observado, no calificación del profesor.

- Análisis técnico: el incremento de Ready con cero consumidores, seguido del consumo del mismo ID tras iniciar los workers, evidencia desacoplamiento temporal: productor y consumidor no necesitan estar activos simultáneamente. Las capturas muestran contadores, no el ID ni el payload dentro del broker; la correlación se sustenta en baseline

vacío, publicación única y logs de API/worker. No se realizó inspección de payload mediante Get messages. Los tiempos son parámetros observados del laboratorio, no umbrales de rúbrica. Se usan timestamps UTC de aplicación, distintos del prefijo Docker; la hora de captura de UI puede corresponder a otra zona y no se equipara a esos eventos. redelivered=false es un dato del log, no garantía exactly-once. Este caso no demuestra caída espontánea, reinicio del broker, CP-12, elasticidad general ni ausencia general de pérdida. La redacción actual es exclusivamente documental: no ejecuta servicios ni altera evidencias originales. El resumen de corrida conserva la historia y sus estados anteriores.

CP-12 — Recuperación de consumidor RabbitMQ

- ID del caso: CP-12, corrida-01, 2026-10-27 UTC.

- Arquitectura: RabbitMQ; API, broker y dos workers originales.

- Objetivo: recuperar el mismo pedido interrumpido antes de confirmación, sin pérdida observada en este ensayo.

- Precondiciones: cola pedidos durable vacía, Consumers=2, ack manual/prefetch=1; API/broker healthy y dos workers running. Contenedores iniciales, baseline, PID1.

- Datos de entrada: POST /orders, customerName=Pedro, product=Libro de arquitectura, quantity=1; ID generado ORD-ea298437-f581-4b8f-b80d-2706f3659b80, correlationId igual. HTTP raw: HTTP 202, 71 ms, una solicitud, cuatro aserciones aprobadas; comando exitcode 0. Aviso de login/cloud no invalida la ejecución local ni acredita publicación en nube.

- Pasos ejecutados:

1. Detener peer worker-2 con cola vacía y esperar Consumers=1.

2. Publicar una vez; worker-1 recibe 03:46:24.274Z, redelivered=false. Suspender PID1 con SIGSTOP y comprobar Unacked=1/Consumers=1 sin processed previo.

3. Ejecutar stop --signal SIGKILL --timeout 0; orden exitcode 0, target exited/137. Esperar Ready=1/Consumers=0 antes de iniciar.

4. Iniciar el mismo target: recibe 03:46:45.109Z, redelivered=true; procesa 03:46:48.123Z (3.014 s). Restaurar peer y verificar cuatro IDs originales.

- Resultado esperado: reentrega/procesamiento del pedido interrumpido y restauración del entorno sin recrear contenedores.

- Resultado obtenido: criterio técnico verificado. API publica 03:46:24.255Z; dos recepciones y un processed del mismo ID en worker-1; API. Target exited/137 en after-stop; final dos workers running/no paused, API/broker healthy, health ready, cola 0/0/0/2 y mismos cuatro IDs. Resumen con comandos y snapshots.

- Evidencia (capturas): Captura real CP-12 aportada por el usuario y revisada completa, de la vista DERIVADA de fuentes originales reales conservadas: worker raw, API raw, Unacked antes del fallo, Ready tras el fallo y restauración final. No es GUI ni log nativo modificado. El PDF muestra un detalle del bloque API–restauración, identificado como recorte de esta vista derivada y enlazado al original completo. Solo excluye tres líneas de encabezado y una de pie; conserva logs, fechas y datos. El original permanece íntegro.

- Estado: Cumple, conclusión propia del reporte, no evaluación del profesor. Cierre técnico y visual de CP-12 RabbitMQ; el registro original conservado documenta la etapa previa a incorporar la imagen y permanece íntegro.

- Análisis técnico: La suspensión SIGSTOP de PID1 fue instrumentación breve, no mensaje confirmado ni caída del broker. El gate Unacked terminó 8.714 s después de la intención STOP (03:46:24.289->03:46:33.003), dentro de 12 s; la orden SIGKILL empezó a los 8.889 s. No es una garantía temporal general. La parada fue manual/controlada en su ejecución, pero abrupta para el proceso (SIGKILL): al desconectarse el consumidor, el broker devuelve el trabajo no confirmado a Ready. No se ensayó recuperación del broker.

Se observan dos entregas y un order_processed del mismo pedido; redelivered=true acredita reentrega, no exactly-once. Un trabajo sin confirmar puede haber producido efectos antes del fallo y duplicarlos al reintentarse; estos logs no prueban ausencia universal de efectos ni idempotencia. Sin pérdida observada para este único ID, no garantía general, HA, downtime cero ni benchmark. Management no contiene el orderId: su asociación depende de baseline vacío, una publicación y logs correlacionados. Los contadores acumulados no representan exclusivamente esta corrida.

Fuente del caso/formato: guía del taller p. 4 (casos y once campos) y Anexo 1 pp. 7–13 (RabbitMQ). Diseño y referencias técnicas: protocolo del ensayo. Conclusiones propias, no calificación del profesor.

Análisis integrado y conclusiones — guía §10

Asincronía, mediación y distribución

CP-07 y CP-11 separan aceptación y consumo: workers ausentes, HTTP 202 en 27 ms, Ready 0 -> 1 y consumo posterior sin publicar de nuevo. No hay SLA. CP-06/08 acreditan el salto por el broker con ID/payload nativos; no basta comparar contadores. CP-10 observa 3/3 en seis pedidos con dos consumidores competidores; no mide escalabilidad lineal, autoscaling ni equilibrio garantizado. CP-09 conserva continuidad de aceptación ante indisponibilidad controlada; no demuestra caída del broker. CP-12 interrumpe trabajo Unacked: vuelve a Ready y se reentrega false -> true; dos entregas y un order_processed del mismo ID, no exactly-once ni idempotencia.

1. ¿Qué dificultades se presentaron en las pruebas?

El stdout de Postman parte UUID en líneas: el parser debe reconstruir el resultado guardado sin publicar pedidos adicionales. El intento --output requirió login cloud; se usó stdout local conservado, no se afirma publicación cloud. Management puede mostrar estadísticas stale: se recurrió a baseline y esperas acotadas, no a una foto como prueba universal. El comando stop terminó 0 mientras los contenedores acabaron 137; son resultados distintos, no parada graciosa. CP-06 necesitó tracing nativo por ID porque stdout operativo del broker no lo contenía. CP-12 requirió instrumentación SIGSTOP y gate Unacked antes de SIGKILL para observar trabajo en vuelo. Fuentes: historial consumidores, trazabilidad, recuperación, reparto.

2. ¿Qué arquitectura es más robusta desde QA?

La evidencia limitada de RabbitMQ respalda desacoplamiento, almacenamiento temporal y recuperación del consumidor en los escenarios descritos. No permite elegir un ganador frente a Kafka, que no fue evaluado en esta entrega. No se prueban HA, downtime cero, ausencia general de pérdida, fallo del broker ni efectos externos exactly-once. Una muestra favorable no es garantía de producción.

3. ¿Qué diferencias reales se evidenciaron entre las arquitecturas?

La comparación experimental sigue pendiente de los diez casos Kafka y análisis conjunto. No se inventan diferencias medidas. Aquí solo se verificaron propiedades

de la arquitectura RabbitMQ descrita en el Anexo 1, sin equivalencia con Kafka.

Fuentes normativas y límites de entrega

Taller Pruebas de Integración.pdf, 24 páginas: p. 3 (componentes independientes, Docker y evidencia de despliegue); p. 4 §7 (casos y once campos); p. 5 §10 (tres preguntas), §11 (documento PDF, pruebas/capturas/logs y repo o ZIP); Anexo 1 pp. 7–13 (flujo RabbitMQ, API, consumo, ack y pruebas). p. 13 complementa la entrega RabbitMQ. Citas según auditoría documental directa de las 24 páginas; no se atribuyen exigencias a una página 22 no identificada por esa auditoría.
No se redistribuye el original del profesor. Los reportes completos anteriores y sus fuentes preservadas son la base experimental, no estas conclusiones como evidencia nueva.
Los enlaces GitHub no acreditan por sí solos publicación remota verificada.
La ejecución aislada PowerShell/Linux documentada pasó; no certifica Windows/Hyper-V nativo, todavía no verificado. Son comprobaciones distintas de esta compilación offline.

Figura 1. CP-01/07/09: captura real de stdout Postman CLI guardado, no GUI. Pedido b7a; aceptación HTTP 202, 27 ms. Fuente corrida 03.

Original preservado

```
POST http://localhost:3000/orders
202 Accepted * 27 ms time * 369 B^ 365 Bv size * 9^ 7v headers * 0 cookies
- ^ raw * 84 B
| {
|   "customerName": "Pedro",
|   "product": "Libro de arquitectura",
|   "quantity": 1
| }
-
- v application/json * text * json * utf8 * 123 B
| {"status":"accepted","message":"Pedido recibido y envi
| ado a RabbitMQ","orderId":"ORD-b7a74952-2957-49cb-a5db
| -f88d7b2ce443"}
-
prepare   wait    dns-lookup   tcp-handshake   transfer-start   download   process   total
4 ms      11 ms   3 ms         1 ms            7 ms             2 ms       875 us    31 ms
```

Figura 2. CP-02/09/11: GUI Management, pendiente con Consumers=0. Contadores sin ID: correlación b7a por baseline vacío y publicación única.

Original preservado

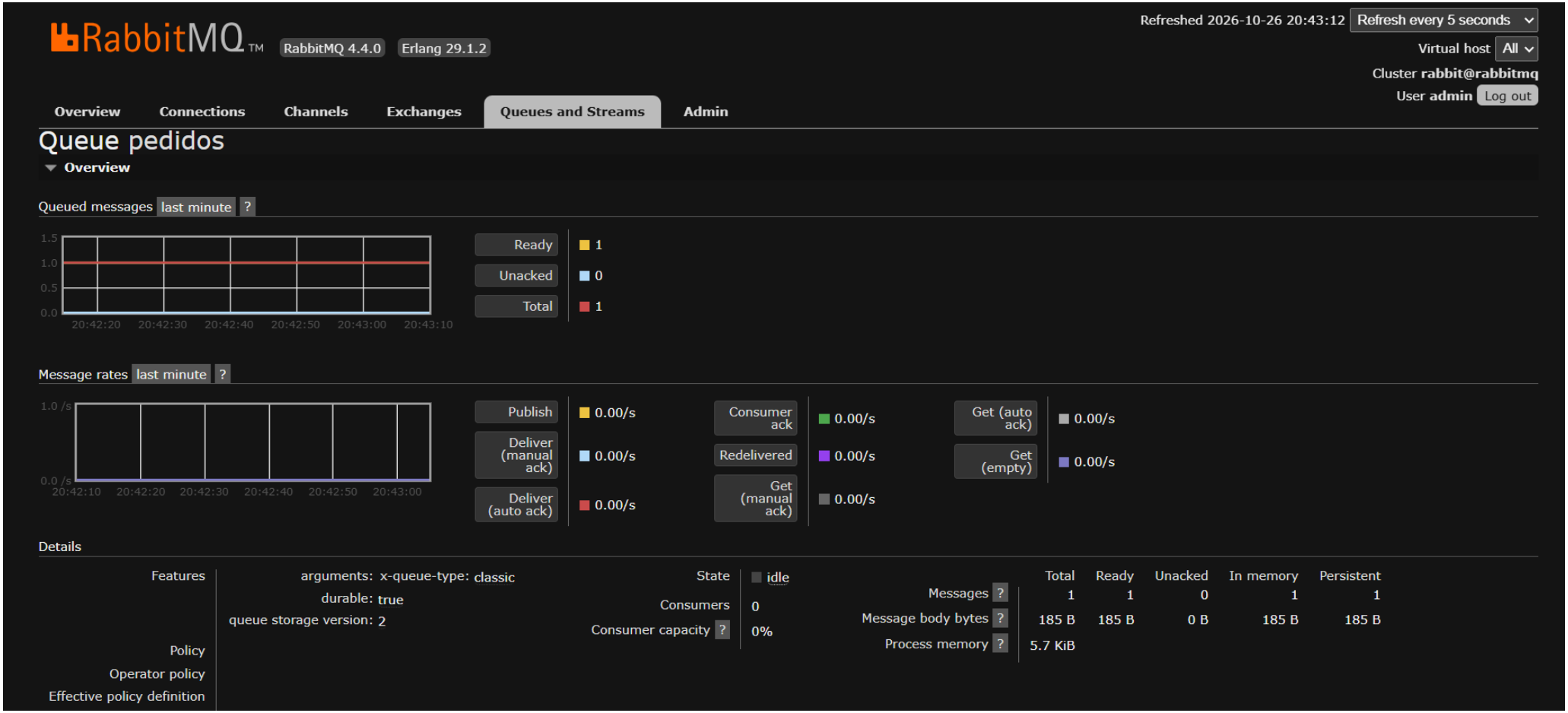


Figura 3. CP-10: Docker Desktop worker-2, tres procesamientos del lote 01:58 UTC. También contiene b7a 01:46 UTC, reutilizado solo para CP-03/07/09/11.

Original preservado

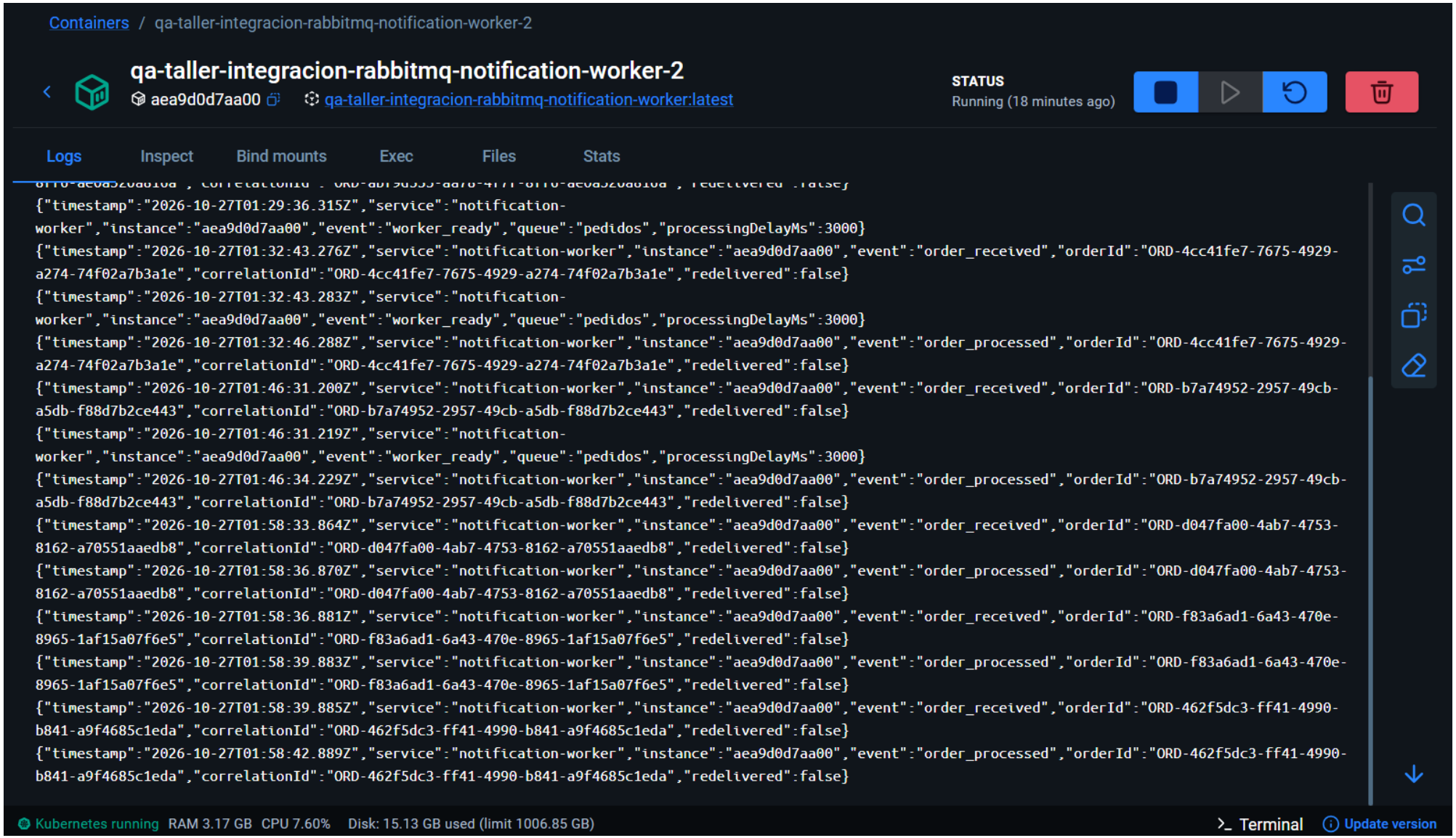


Figura 4. CP-03/09/11: GUI Management final, cola vacía y dos consumidores. No muestra ID: contrastar logs b7a de corrida 03.

Original preservado

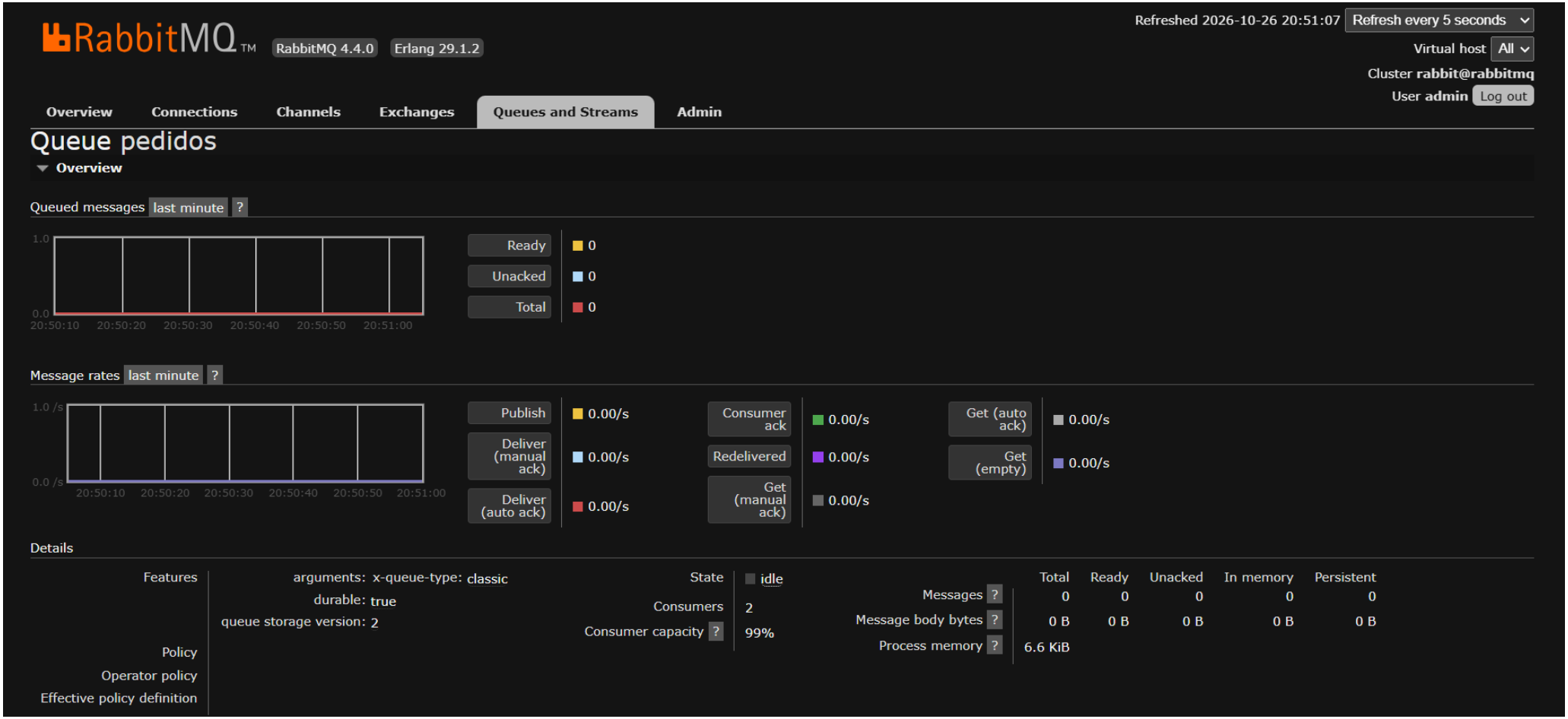


Figura 5. CP-06/08, ID 8d48: captura real de VISTA DERIVADA de registros reales. No GUI ni JSONL nativo; consultar broker-native-original.jsonl. Sin recortes.

Original preservado

```
CP-06 RABBITMQ | VISTA DERIVADA DE REGISTROS REALES
No es captura GUI ni log nativo nuevo. No se ejecuta otra solicitud.
Fuentes originales conservadas en esta misma carpeta.

API: logs-orders-api-same-id.txt
2026-10-27T02:42:35.123Z  order_published
orderId:       ORD-8d488fb8-62c9-4e37-935d-31aed2a8aa9e
correlationId: ORD-8d488fb8-62c9-4e37-935d-31aed2a8aa9e

BROKER NATIVO: broker-native-original.jsonl
2026-10-27T02:42:35.087+00:00  published  nodo=rabbit@rabbitmq
correlation_id: ORD-8d488fb8-62c9-4e37-935d-31aed2a8aa9e
message_id:     ORD-8d488fb8-62c9-4e37-935d-31aed2a8aa9e
payload.orderId (base64 decodificado): ORD-8d488fb8-62c9-4e37-935d-31aed2a8aa9e
routing_keys=["pedidos"]; queue=none
2026-10-27T02:42:35.103+00:00  received  nodo=rabbit@rabbitmq
correlation_id: ORD-8d488fb8-62c9-4e37-935d-31aed2a8aa9e
message_id:     ORD-8d488fb8-62c9-4e37-935d-31aed2a8aa9e
payload.orderId (base64 decodificado): ORD-8d488fb8-62c9-4e37-935d-31aed2a8aa9e
routing_keys=["pedidos"]; queue=pedidos

WORKER-1: logs-notification-worker-same-id.txt
2026-10-27T02:42:35.158Z  order_received
orderId:       ORD-8d488fb8-62c9-4e37-935d-31aed2a8aa9e
correlationId: ORD-8d488fb8-62c9-4e37-935d-31aed2a8aa9e
2026-10-27T02:42:38.411Z  order_processed
orderId:       ORD-8d488fb8-62c9-4e37-935d-31aed2a8aa9e
correlationId: ORD-8d488fb8-62c9-4e37-935d-31aed2a8aa9e

Postman: HTTP 202; 1 solicitud; 4/4 aserciones; exitcode=0.
Limpieza verificada: tracing OFF; 2 workers; cola 0/0/0.
Vista con campos seleccionados. JSON y stdout originales no modificados.
```

Figura 6. CP-10: Docker Desktop worker-1, lote 01:58 UTC, tres procesamientos. No confundir registros previos de otros pedidos.

Original preservado

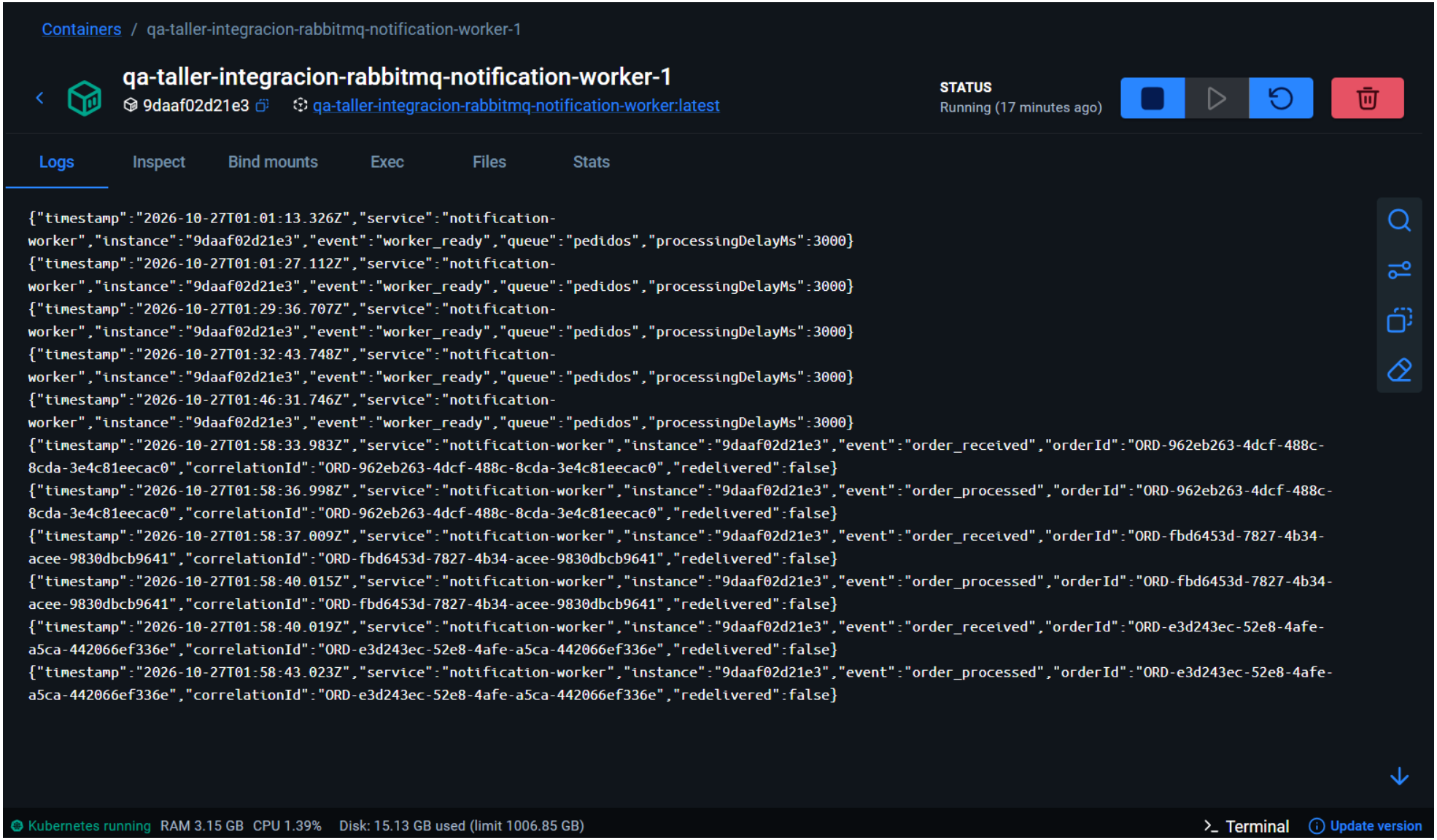


Figura 7. CP-12, ID ea298: captura real de VISTA DERIVADA de fuentes originales. No GUI. Detalle de captura; original completo enlazado. Recorte: solo tres líneas de encabezado y una de pie; logs y fechas conservados. Dos entregas, un processed; no exactly-once.

Original preservado

```
API [final-logs-28a4ac996bc2.txt / 027.stdout.txt]
03:46:24.255Z order_published; HTTP 202, 71 ms, 1 request, 4/4
ORD-ea298437-f581-4b8f-b80d-2706f3659b80
WORKER-1 target 9daaf02d21e3 [final-logs-9daaf02d21e3.txt]
03:46:24.274Z order_received redelivered=false
ORD-ea298437-f581-4b8f-b80d-2706f3659b80
INSTRUMENTACION [028.command.json / intent.jsonl]
03:46:24.289Z docker kill --signal SIGSTOP target; exitcode=0
PID1 suspendido brevemente; NO ack ni caida del broker.
ANTES [before-fault-queue.json / 061.get.json]
03:46:32.993-33.003Z Ready=0 Unacked=1 Total=1 Consumers=1
FALLO [064.command.json / after-stop-containers.json]
03:46:33.178Z docker stop --signal SIGKILL --timeout 0 target
Orden exitcode=0; target exited, ExitCode=137, OOMKilled=false
REQUEUE [after-fault-queue.json / 103.get.json]
03:46:42.944Z Ready=1 Unacked=0 Total=1 Consumers=0
REINICIO [104.command.json] docker start mismo target; exitcode=0
REENTREGA [final-logs-9daaf02d21e3.txt]
03:46:45.109Z order_received redelivered=true
ORD-ea298437-f581-4b8f-b80d-2706f3659b80
PROCESADO [final-logs-9daaf02d21e3.txt]
03:46:48.123Z order_processed redelivered=true (3.014 s)
ORD-ea298437-f581-4b8f-b80d-2706f3659b80
RESTAURACION [117.command.json] docker start peer; exitcode=0
FINAL [final.json / 165.get.json / original-containers.json]
03:46:52.947Z Ready=0 Unacked=0 Total=0 Consumers=2
Mismos 4 IDs; 2 workers running/no paused; API/broker healthy
API 28a4ac996bc2; broker 874ff8f0d7a7
Workers 9daaf02d21e3 / aea9d0d7aa00; IDs completos en originales
[recovery.json] 1 publicacion, cleanup=[], fin 03:46:53.157Z
Dos entregas / un processed observado; NO exactly-once general.
```